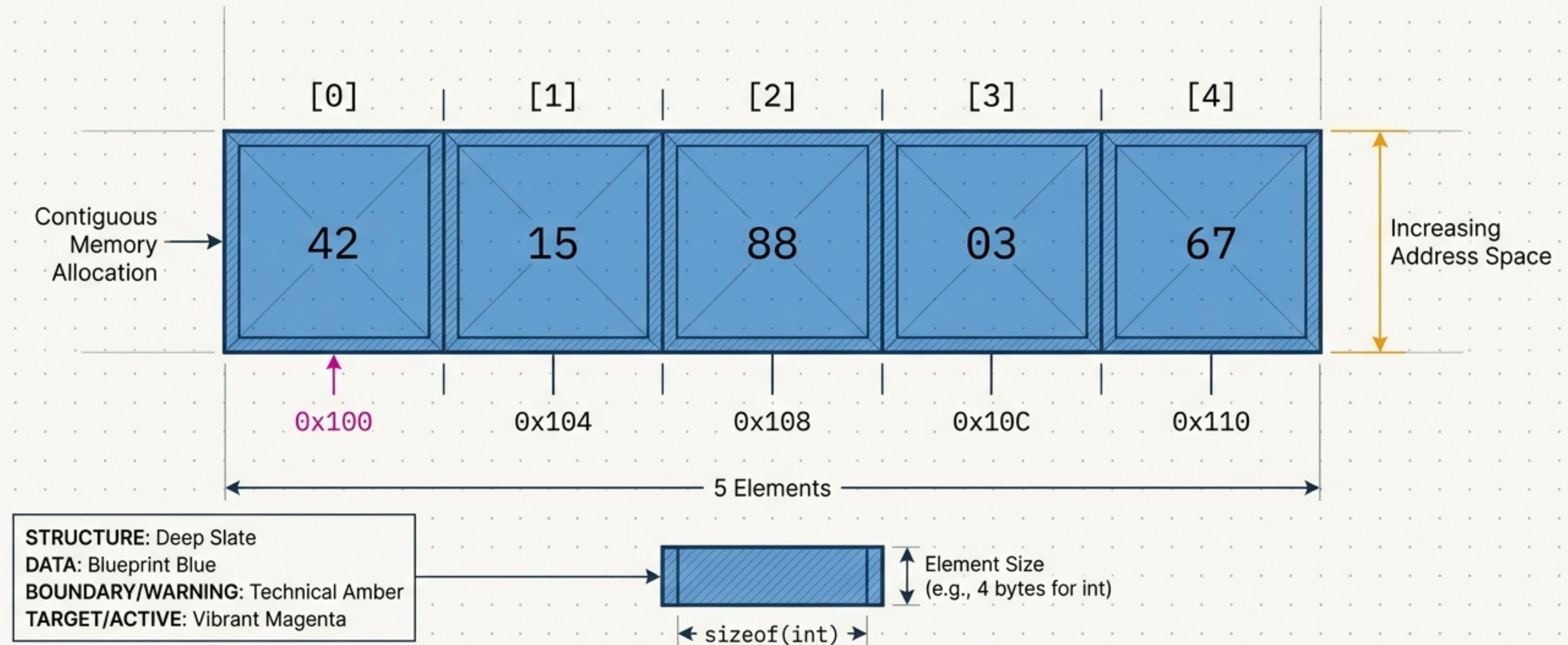


Array Data Structures in C++

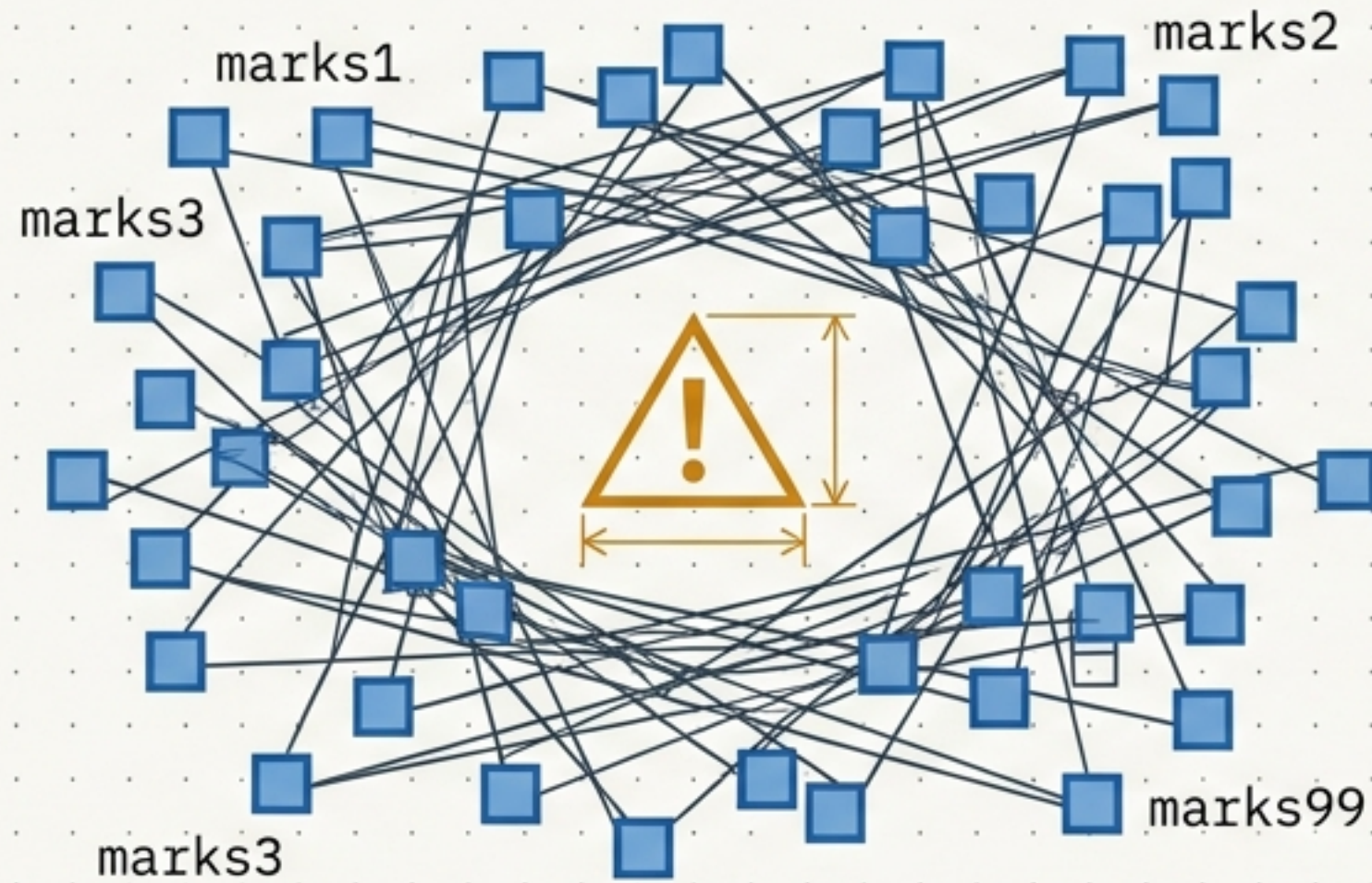
The Fundamental Building Block of Memory and Algorithms



The Scaling Problem

100 Independent Variables

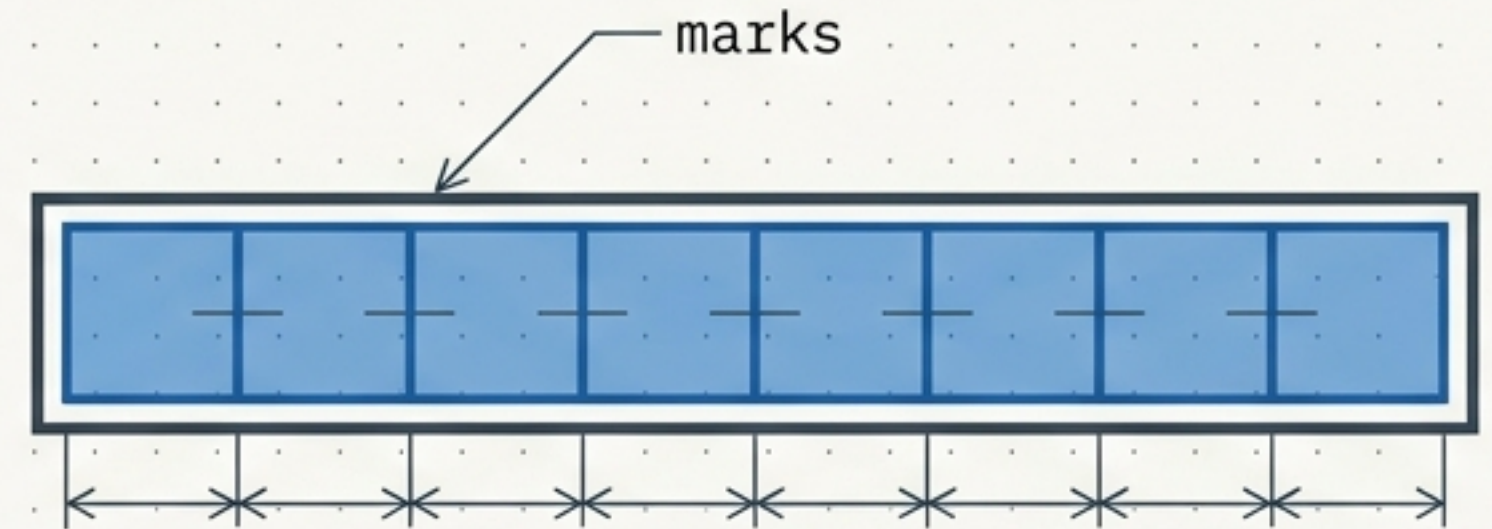
Hard to create, impossible to track.



Hard to create, impossible to track.

1 Array Data Structure

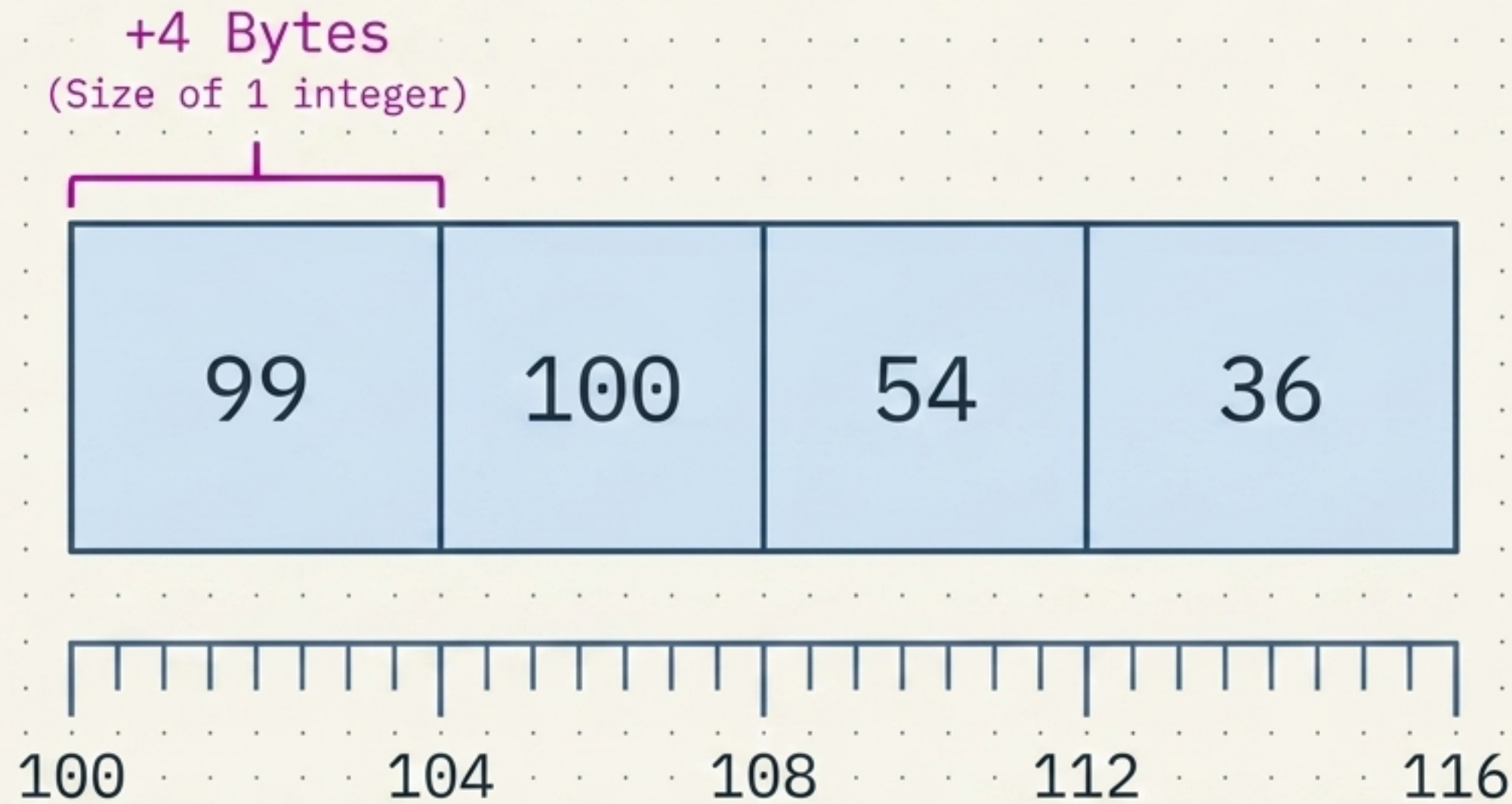
A unified block tracked by a single name.



A unified block tracked by a single name.

The Memory Blueprint: Contiguous Allocation

Arrays are contiguous and linear. Data is stored in an unbroken sequence of memory, jumping forward by the exact byte-size of the data type.



Code to Canvas: Initialization

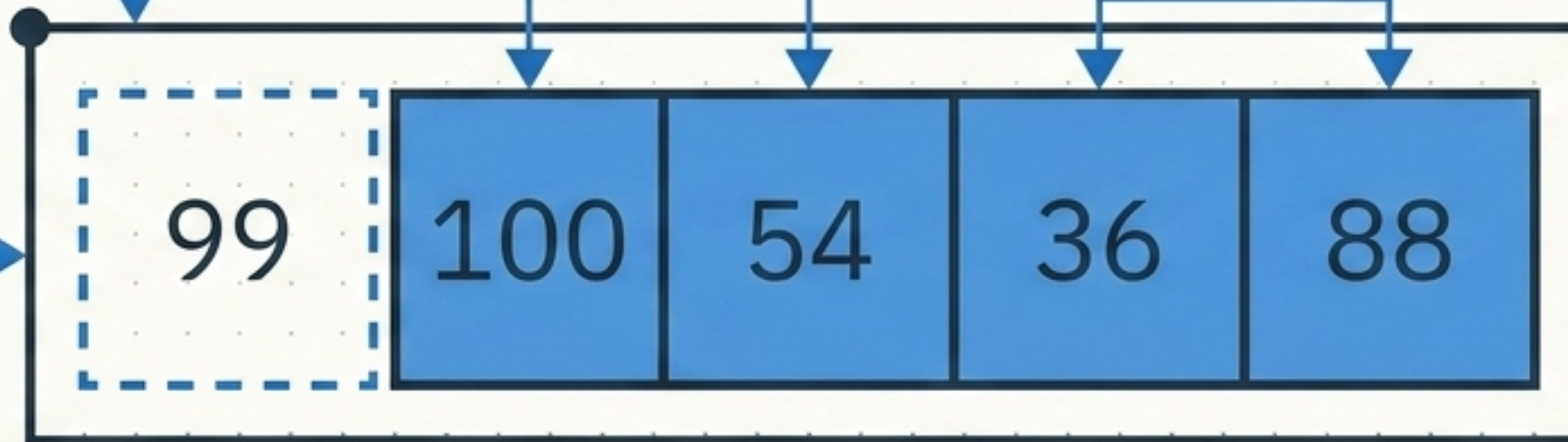
```
int marks[5] = {99, 100, 54, 36, 88};
```

Data Type constraint
(Same type only)

The single tracked
variable name

Total Size capacity

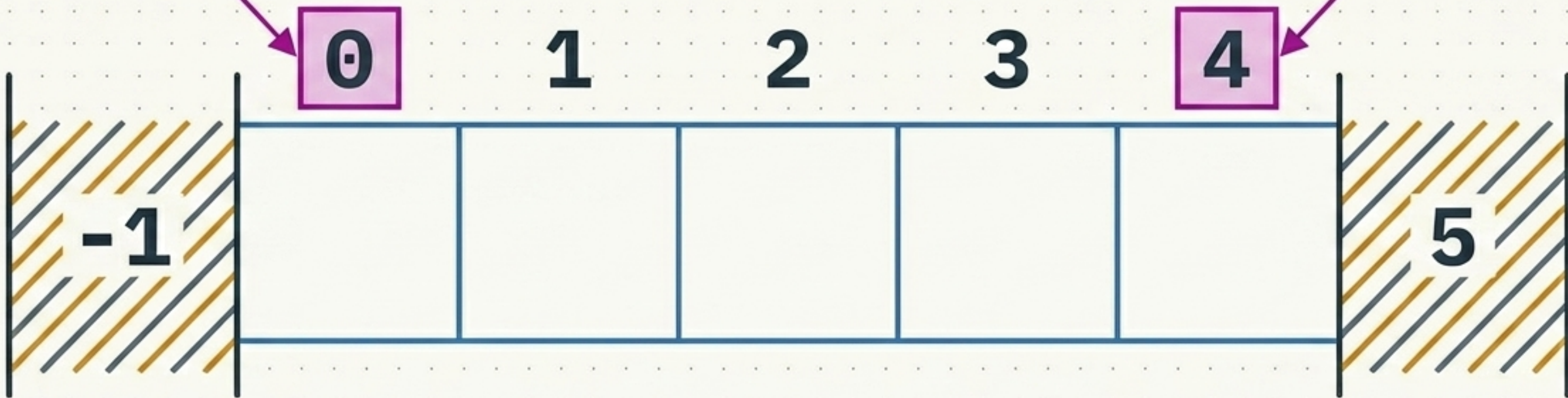
Injected Values



The Zero-Index Law

Always starts at Zero.

Size - 1



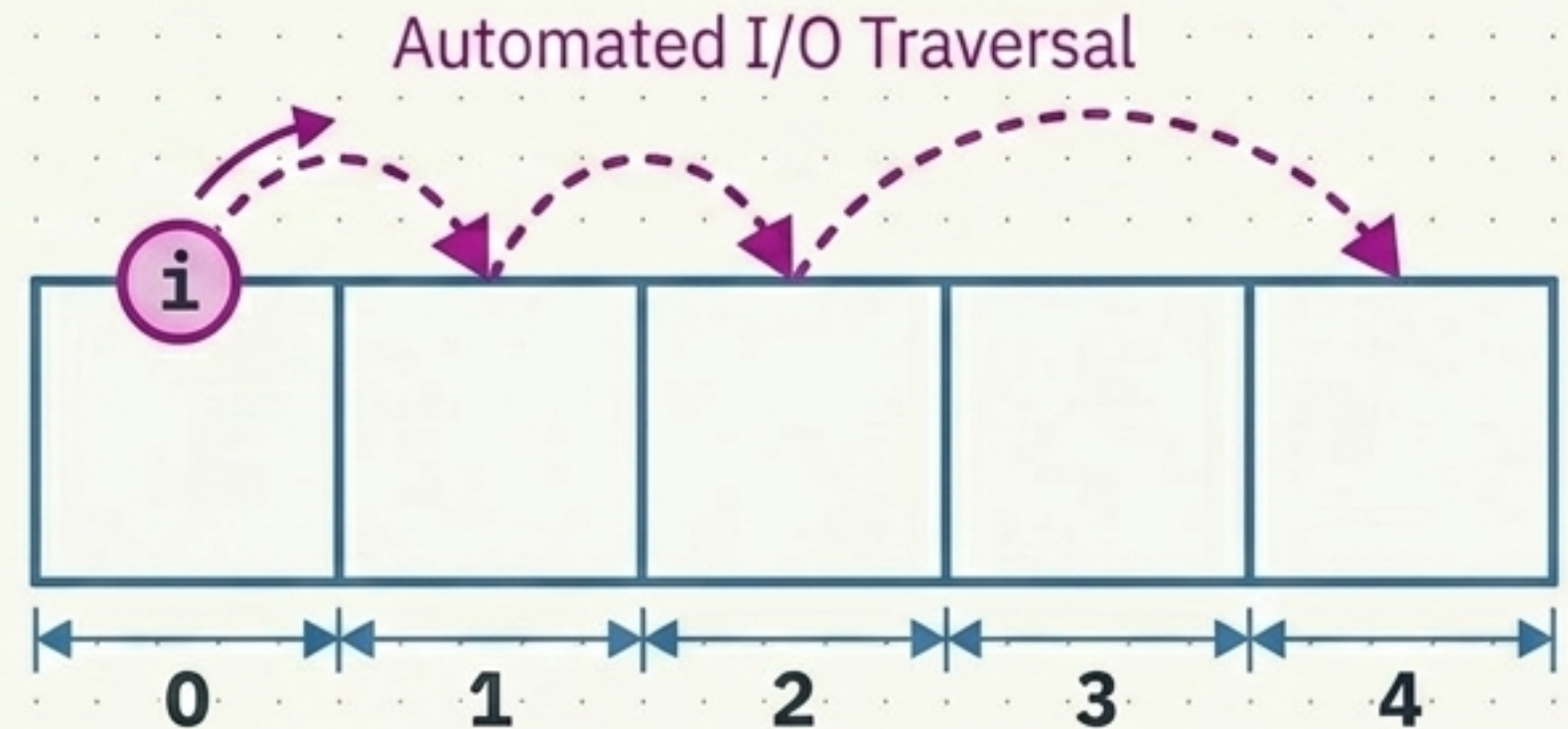
Accessing invalid indices fetches garbage values and triggers **out-of-bounds memory errors**.

Dynamic Sizing and Loop Traversal

$$\frac{\text{sizeof(marks)}}{\text{sizeof(int)}} = 5$$

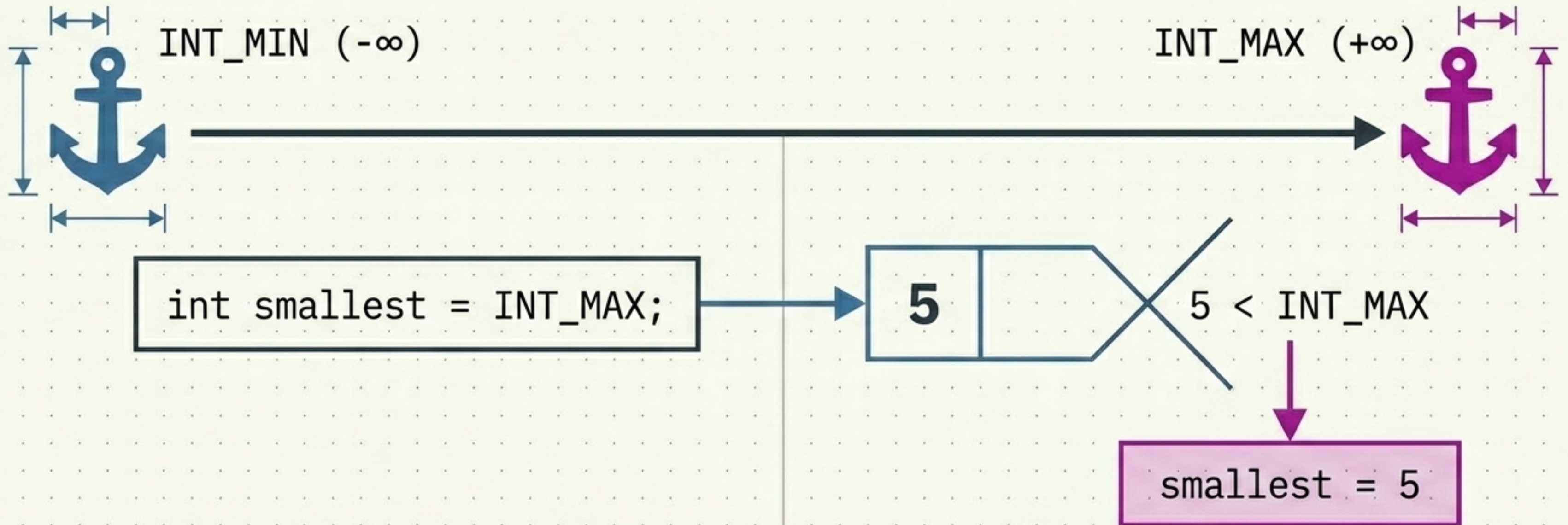
(Total footprint: 20 Bytes) / (Single block: 4 Bytes) = 5 (Total Elements)

```
for(int i = 0; i < size; i++) {  
    cout << marks[i];  
}
```



Algorithm I: Find Min & Max

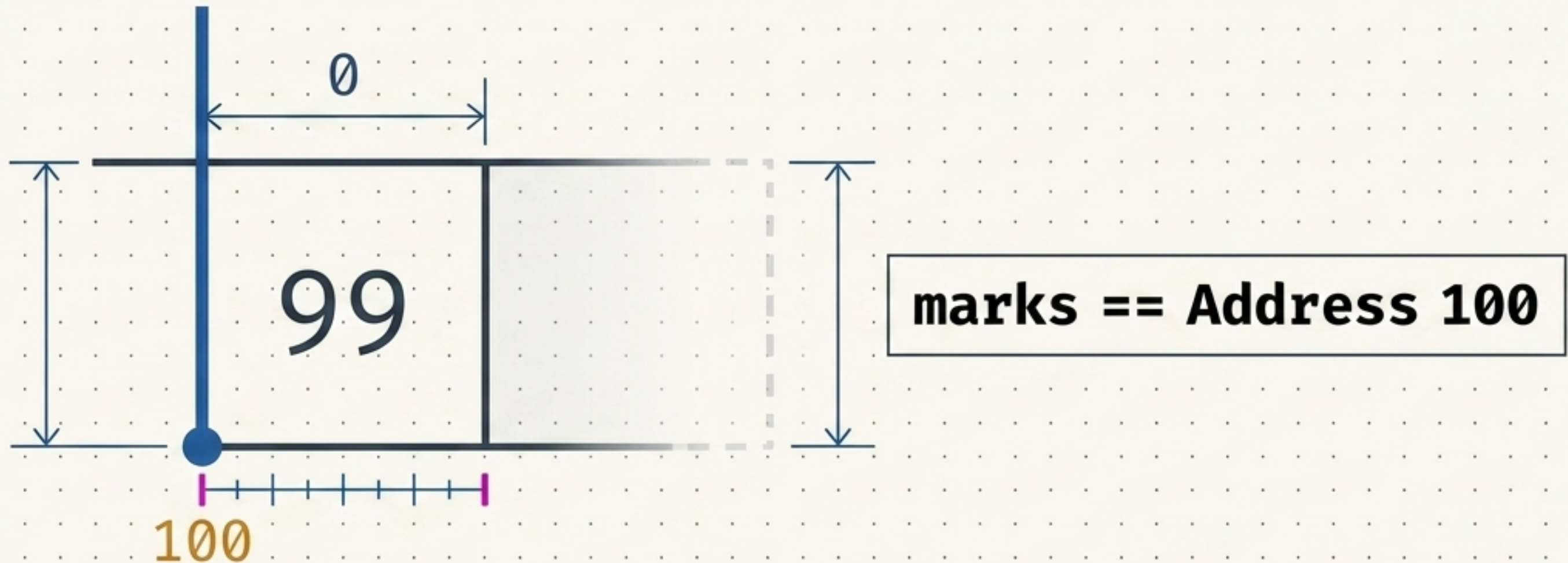
By anchoring our comparison variables to the furthest possible extremes, the very first element is guaranteed to trigger a replacement.



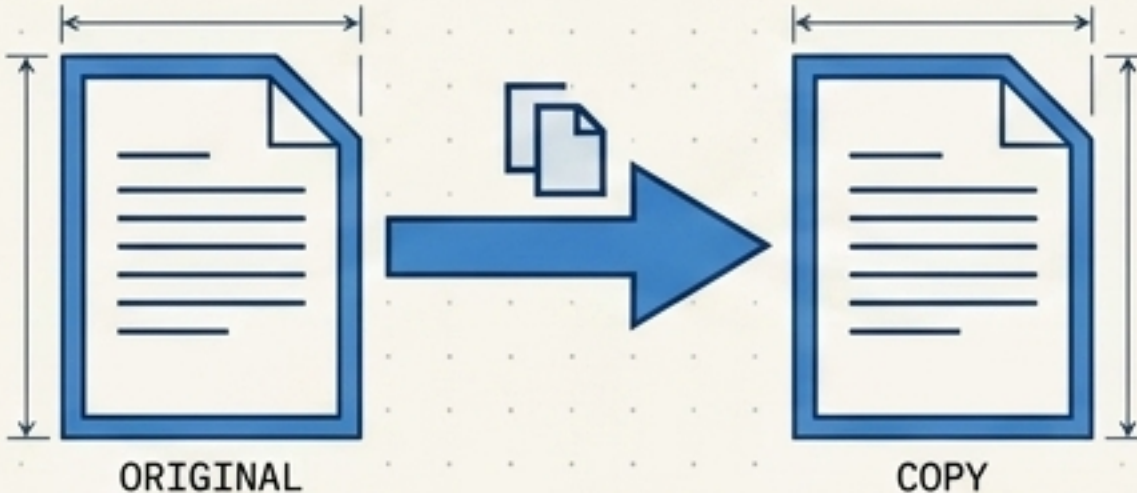
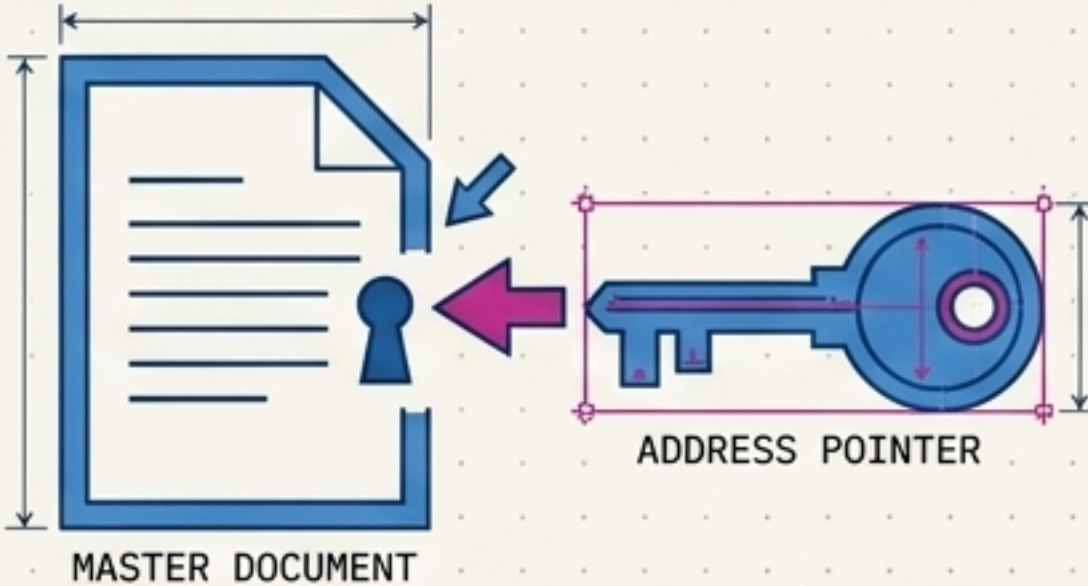
Under the Hood: The Implicit Pointer

In C++, the array's name acts implicitly as a pointer. It does not hold the whole array; it holds the specific memory address of the first block. From there, the system just steps forward.

marks

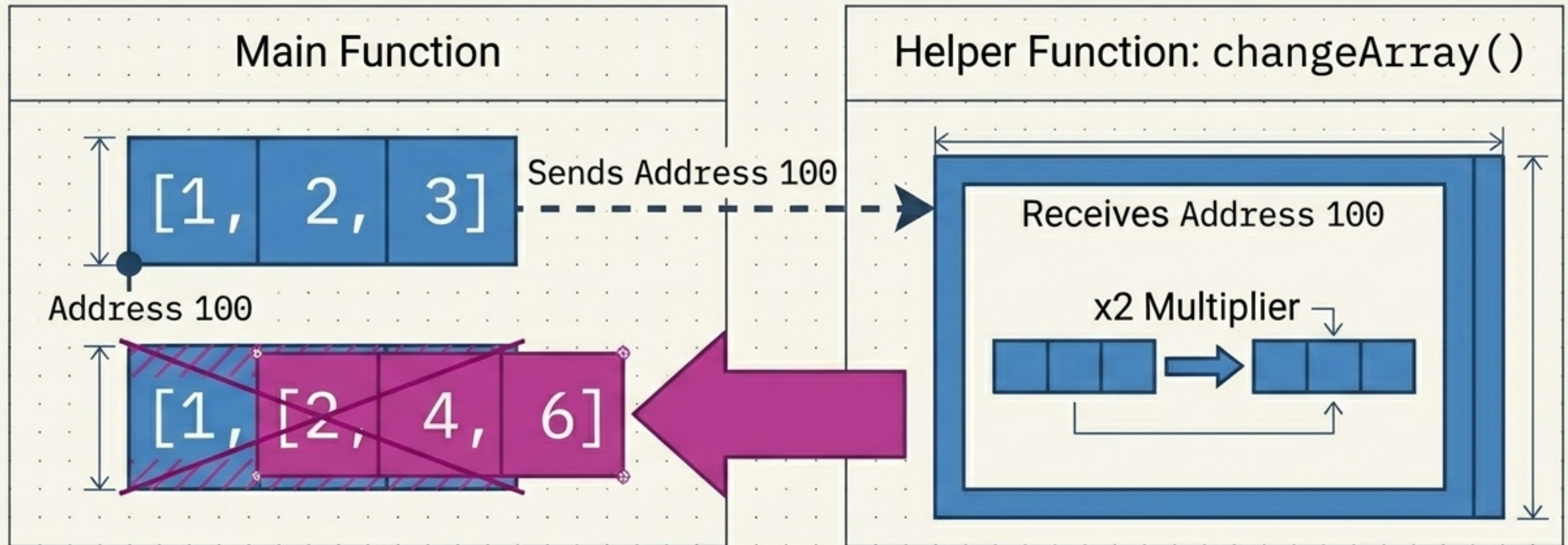


Pass by Value vs. Pass by Reference

	Pass by Value (Primitive Types)	Pass by Reference (Arrays)
Mechanism	Creates a full, isolated clone in a new memory address.	Passes only the starting address (the implicit pointer).
Memory Footprint	High (duplicates data).	Low (just sends an address).
Side Effects	Changes isolated to the function. Original variable is safe.	Changes immediately mutate the original global state.
Visual Icon	 ORIGINAL COPY	 MASTER DOCUMENT ADDRESS POINTER

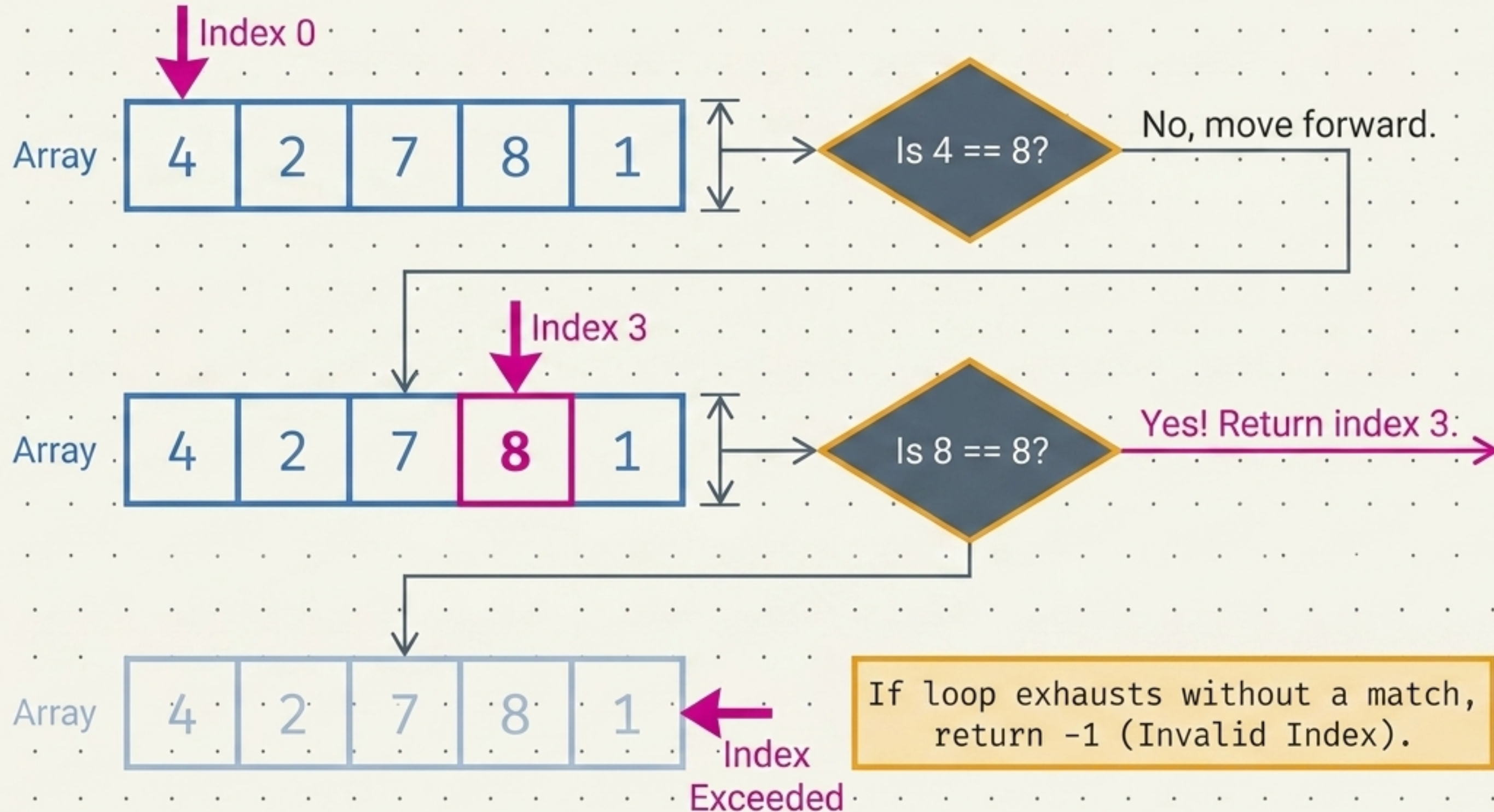
The Chain Reaction of Reference

Because the function was handed the map (address) rather than a copy of the treasure, modifications instantly rewrite the original data.



Algorithm II: Linear Search Mechanics

Target Value = 8



Linear Search in Code

Time Complexity: $O(N)$

Linear Time - Execution scales identically with array size.

```
int linearSearch(int arr[], int size, int target) {  
    for (int i = 0; i < size; i++) {  
        if (arr[i] == target) {  
            return i;  
        }  
    }  
    return -1;  
}
```

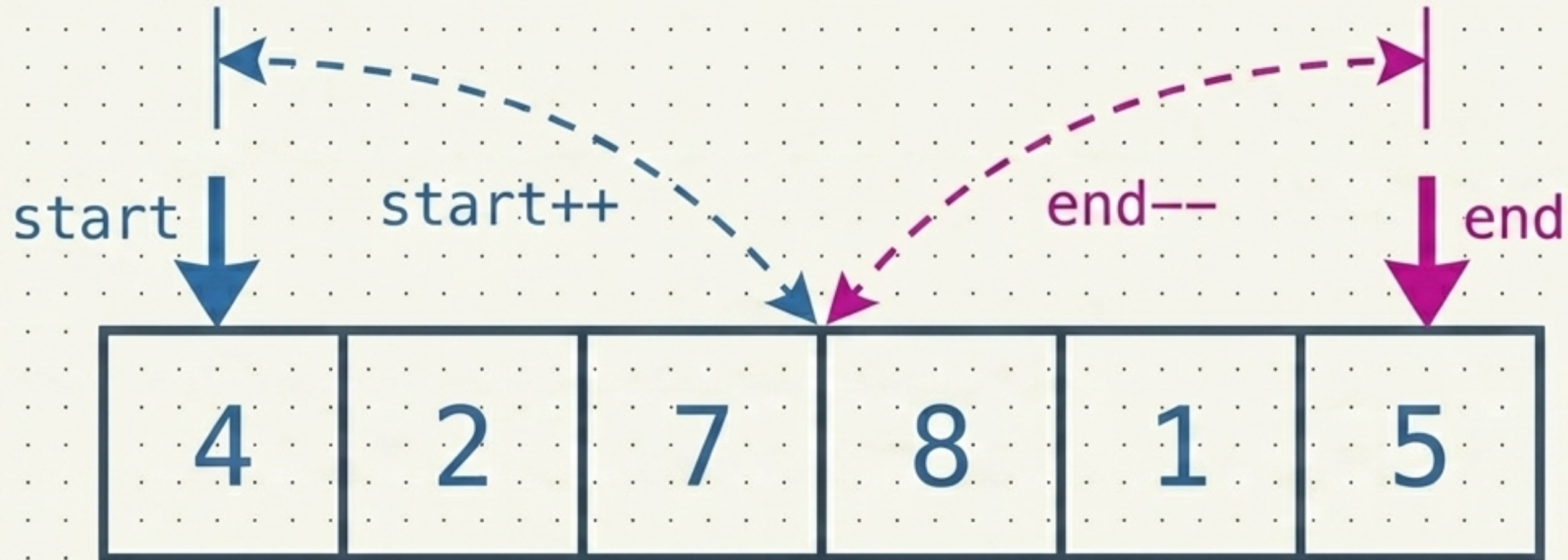
Immediate Success:
Breaks the loop early.

Fallback: Only executes
if the entire loop fails.

Algorithm III: Reversing an Array

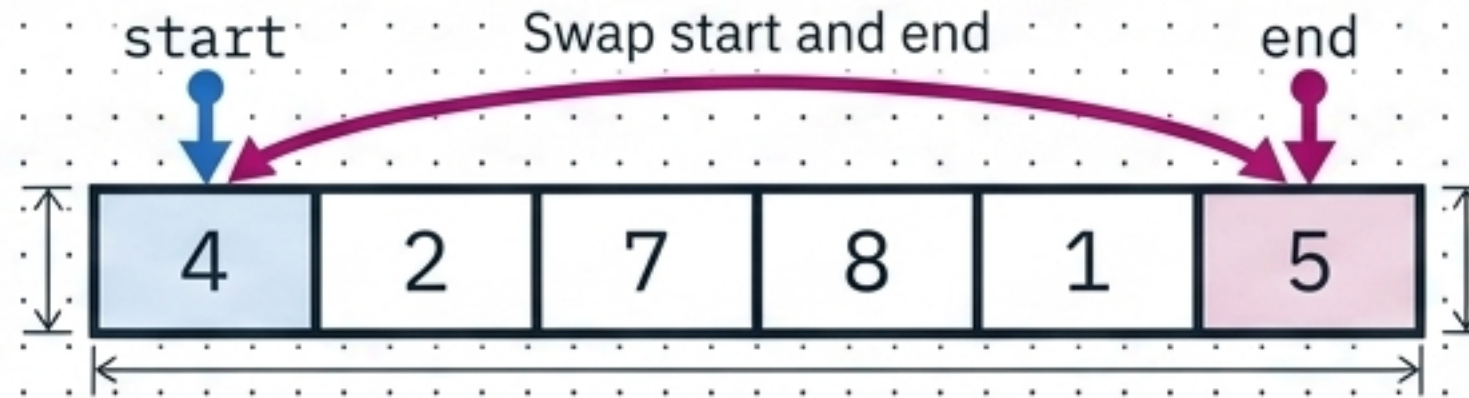
Instead of building a second array, we utilize two points of reference marching toward the center, swapping extremes in-place.

The Two-Pointer Pincer

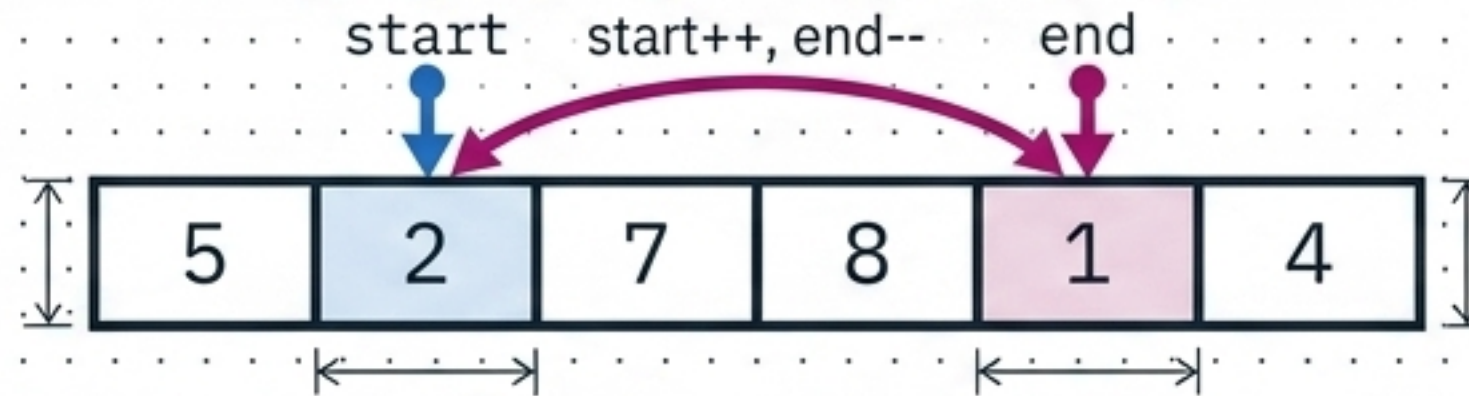


The Two-Pointer Execution

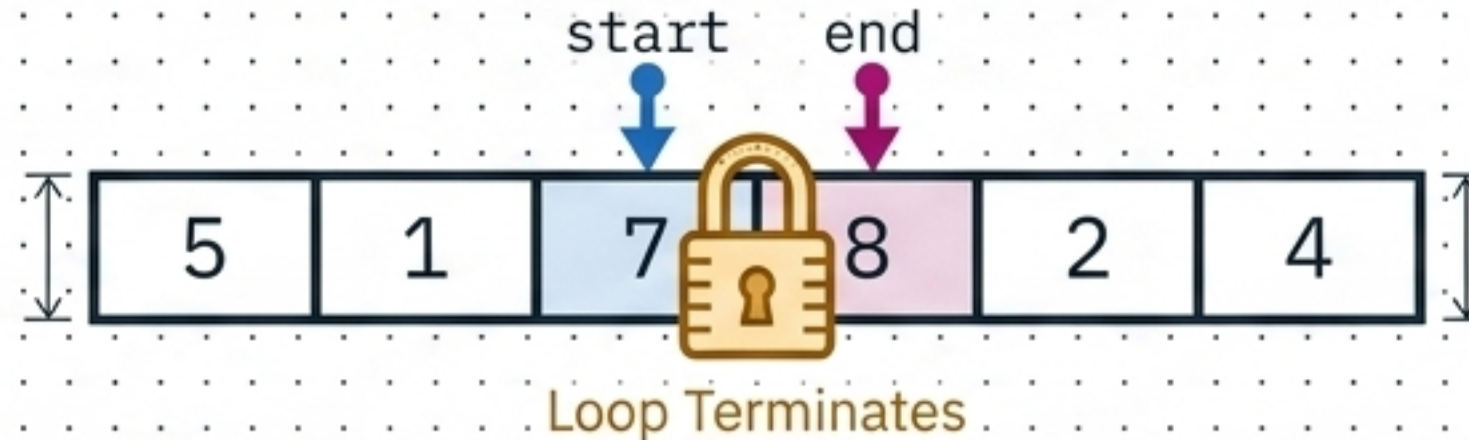
Step 1: Outer Swap



Step 2: Inner Swap



Step 3: Collision / Stop



```
while (start < end)
```

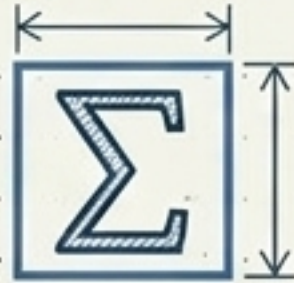
If start crosses or equals end, the reversal is complete.
Continuing would re-reverse the array back to the original.

Time Complexity: $O(N)$

Synthesis & Next Steps

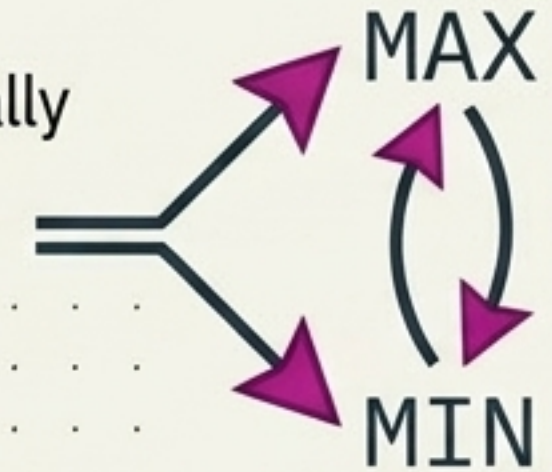
The Basics (Homework)

Challenge: Calculate the Sum and Product of all numeric elements in a given array using a single traversal loop.



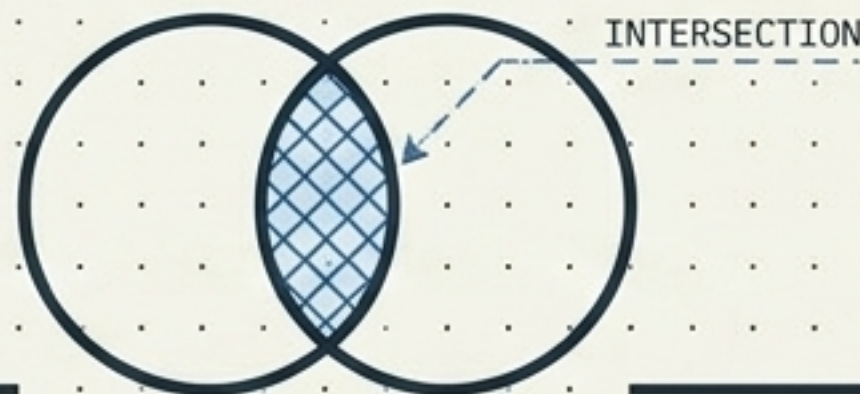
The Logic (Homework)

Challenge: Find and mathematically swap the absolute Minimum and Maximum values within an array in-place.



The Nested Loop (Homework)

Challenge: Print all uniquely occurring values, and calculate the intersection (shared elements) between two distinct arrays.



Coming Next: C++ Vectors

Transitioning from fixed-size arrays to dynamic, auto-scaling memory structures required for LeetCode, CodeForces, and modern systems engineering.

